

LAPORAN PRAKTIKUM PBO

Ekspansi Produk Keuangan (Inheritance)

Disusun Oleh:

Endy Pardilian 2511531017

Dosen Pengampu:

NURFIAH, S.ST, M.Kom

Asisten Pratikum:

EzzaAddini



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

LangkahPengerjaan:

A. Praktikum

1. Class Rekening

```
package pekan4;
import java.util.ArrayList;
public class Rekening {
    String nomorRekening;
    String namaPemilik;
    double saldo;
    private String pin;

    ArrayList<Transaksi> riwayatTransaksi;

    public static boolean isValidPin(String pin) {
        if (pin == null || pin.length() != 6) return false;
        for (int i = 0; i < pin.length(); i++) {
            if (!Character.isDigit(pin.charAt(i))) return false;
        }

        for (int i = 0; i < pin.length(); i++) {
            for (int j = i + 1; j < pin.length(); j++) {
                if (pin.charAt(i) == pin.charAt(j)) {
                    return false;
                }
            }
        }

        for (int i = 0; i < pin.length() - 1; i++) {
            int selisih = Math.abs(pin.charAt(i) - pin.charAt(i
+ 1));
            if (selisih == 1) {
                return false;
            }
        }

        return true;
    }

    public Rekening (String nomor, String nama, double saldoAwal,
String pinAwal) {
        this.nomorRekening = nomor;
        this.namaPemilik = nama;
        this.saldo = saldoAwal;

        if (isValidPin(pinAwal)) {
            this.pin = pinAwal;
        }else {
            System.out.println("Peringatan: PIN tidak valid!
Menggunakan PIN default 804927");
            this.pin = "804927";
        }

        this.riwayatTransaksi = new ArrayList<>();
    }
}
```

```

        System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat.");
    }

    public String getNomorRekening() { return nomorRekening;}
    public String getNamaPemilik() { return namaPemilik;}

    public boolean otentikasi(String inputPin) {
        return this.pin.equals(inputPin);
    }

    public void setorTunai(double nominal) {
        if (nominal > 0 && nominal <= 500000 ) {
            saldo += nominal;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Kredit",
nominal);
            riwayatTransaksi.add(trxBaru);

            System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
        }else {
            System.out.println("Gagal: nominal setor harus lebih
dari 0 atau melebihi 500.000!");
        }
    }

    public void tarikTunai (double nominalDebit) {

        if (nominalDebit > 10000) {
            if(saldo < nominalDebit) {
                System.out.println("saldo anda tidak cukup");

            }else {
                saldo -= nominalDebit;
                String idTrx = "TRX-S-" + System.currentTimeMillis();
                Transaksi trxBaru = new Transaksi (idTrx, "Debit",
nominalDebit);
                riwayatTransaksi.add(trxBaru);
                System.out.println("tarik tunai Rp"+ nominalDebit +
"berhasil. Saldo saat ini : Rp"+saldo);
            }

        }else {
            System.out.println("Gagal: Nominal tarik harus lebih
dari Rp10000!");
        }
    }

    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
            System.out.println(" Belum ada transaksi");
        }else {
            for (Transaksi trx : riwayatTransaksi) {
                trx.cetakDetail();
            }
        }
    }

```

```

    }
}

public void cekInformasi() {
    System.out.println("--- INFO REKENING ---");
    System.out.println("No. Rekening: " + nomorRekening);
    System.out.println("Nama Pemilik: " + namaPemilik);
    System.out.println("Saldo Akhir: Rp" + saldo);
    System.out.println("-----");
    System.out.println("");
}
}

```

Class Rekening tetap berfungsi sebagai blueprint sebuah rekening bank, namun pada versi ini ditambah atribut PIN yang bersifat private dan daftar riwayat transaksi.

a. public static boolean isValidPin

```

public static boolean isValidPin(String pin) {
    if (pin == null || pin.length() != 6) return false;
    for (int i = 0; i < pin.length(); i++) {
        if (!Character.isDigit(pin.charAt(i))) return false;
    }

    for (int i = 0; i < pin.length(); i++) {
        for (int j = i + 1; j < pin.length(); j++) {
            if (pin.charAt(i) == pin.charAt(j)) {
                return false;
            }
        }
    }

    for (int i = 0; i < pin.length() - 1; i++) {
        int selisih = Math.abs(pin.charAt(i) - pin.charAt(i
+ 1));
        if (selisih == 1) {
            return false;
        }
    }

    return true;
}

```

Method ini memeriksa apakah sebuah PIN memenuhi aturan keamanan dan mengembalikan true jika valid atau false jika tidak. Pemeriksaan dilakukan dalam tiga tahap secara berurutan:

1. Panjang dan jenis karakter: PIN tidak boleh null, harus tepat 6 karakter, dan setiap karakternya harus angka (dicek dengan `Character.isDigit`).
2. Tidak ada angka berulang: perulangan bersarang membandingkan setiap karakter pada posisi `i` dengan seluruh karakter di posisi setelahnya (`j = i + 1`). Jika ada dua karakter yang sama, PIN ditolak.
3. Tidak ada angka berurutan: setiap dua karakter yang bersebelahan dihitung selisih nilai absolutnya. Jika selisihnya 1 (misalnya 1 dan 2, atau 5 dan 4), PIN ditolak.

b. `public Rekening`

```
public Rekening (String nomor, String nama, double saldoAwal, String
pinAwal) {
    this.nomorRekening = nomor;
    this.namaPemilik = nama;
    this.saldo = saldoAwal;

    if (isValidPin(pinAwal)) {
        this.pin = pinAwal;
    } else {
        System.out.println("Peringatan: PIN tidak valid!
Menggunakan PIN default 804927");
        this.pin = "804927";
    }

    this.riwayatTransaksi = new ArrayList<>();

    System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat.");
}
```

Constructor kini menerima empat parameter, yaitu nomor rekening, nama pemilik, saldo awal, dan PIN awal. Setelah menyimpan tiga data pertama, constructor memvalidasi PIN dengan `isValidPin()`. Jika valid, PIN disimpan; jika tidak, sistem memberikan peringatan dan memakai PIN default 804927 sebagai pengaman agar objek tidak pernah berada dalam kondisi tanpa PIN. Constructor juga membuat `ArrayList` kosong untuk riwayatTransaksi karena rekening yang baru dibuat belum memiliki transaksi. Berbeda dengan versi pekan pertama, pesan konfirmasi sekarang tidak lagi menampilkan saldo.

c. `public void setorTunai`

```
public void setorTunai(double nominal) {
    if (nominal > 0 && nominal <= 500000 ) {
        saldo += nominal;
        String idTrx = "TRX-S-" + System.currentTimeMillis();
        Transaksi trxBaru = new Transaksi (idTrx, "Kredit",
nominal);
        riwayatTransaksi.add(trxBaru);
    }
}
```

```

        System.out.println("Setor tunai Rp" + nominal + "
berhasil. Saldo saat ini: Rp" + saldo);
    }else {
        System.out.println("Gagal: nominal setor harus lebih
dari 0 atau melebihi 500.000!");
    }
}

```

Method ini memproses penyetoran uang. Setoran diterima jika nominalnya lebih dari 0 dan tidak melebihi Rp500.000. Jika lolos validasi, saldo ditambah, lalu dibuat ID transaksi unik dari awalan “TRX-S-“ digabung waktu saat ini dalam milidetik (System.currentTimeMillis()). Selanjutnya dibuat objek Transaksi berjenis “Kredit” dan ditambahkan ke riwayatTransaksi sebelum pesan keberhasilan dicetak. Jika validasi gagal, saldo dan riwayat tidak berubah sama sekali dan hanya pesan kegagalan yang ditampilkan.

d. public void tarikTunai

```

public void tarikTunai (double nominalDebit) {

    if (nominalDebit > 10000) {
        if(saldo < nominalDebit) {
            System.out.println("saldo anda tidak cukup");

        }else {
            saldo -= nominalDebit;
            String idTrx = "TRX-S-" + System.currentTimeMillis();
            Transaksi trxBaru = new Transaksi (idTrx, "Debit",
nominalDebit);
            riwayatTransaksi.add(trxBaru);
            System.out.println("tarik tunai Rp"+ nominalDebit +
"berhasil. Saldo saat ini : Rp"+saldo);
        }

    }else {
        System.out.println("Gagal: Nominal tarik harus lebih
dari Rp10000!");
    }
}

```

Method ini memproses penarikan uang dengan dua lapis validasi. Lapis pertama memastikan nominal penarikan lebih dari Rp10.000. Lapis kedua memastikan saldo mencukupi; bila saldo lebih kecil daripada nominal, penarikan ditolak dengan pesan “saldo anda tidak cukup”. Hanya jika kedua syarat terpenuhi, saldo dikurangi, dibuat objek Transaksi berjenis “Debit”, dan objek tersebut disimpan ke riwayatTransaksi. Dengan struktur ini saldo tidak akan pernah bernilai negatif.

e. public void cetakMutasi

```

public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println(" Belum ada transaksi");
    }else {

```

```

        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
    }
}

```

Method ini menampilkan seluruh riwayat transaksi rekening. Jika riwayat Transaksi kosong, ditampilkan pesan bahwa belum ada transaksi. Jika berisi, program melakukan perulangan for-each pada setiap objek Transaksi dan memanggil cetakDetail() milik objek tersebut. Ini adalah contoh interaksi antarobjek. Rekening tidak mencetak data transaksi sendiri, melainkan meminta tiap Transaksi mencetak rinciannya.

2. Class Main

```

package pekan4;
import java.util.Scanner;
public class Main {
    public static void main (String[]args) {
        Scanner input = new Scanner (System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;
        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun Aktif");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Simulasi akhir Bulan (Khusus
Tabungan");

            System.out.println("0. Keluar");
            System.out.println("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine();
            switch (pilihan) {

                case 1 :
                    System.out.println("Masukkan No Rekening: ");
                    String no = input.nextLine();
                    System.out.println("Masukkan Nama Pemilik: ");
                    String nama = input.nextLine();
                    System.out.println("Masukkan Saldo Awal: ");
                    double saldo = input.nextDouble();
                    input.nextLine();

```

```

        System.out.println("Masukkan PIN (6 digit angka): ");
        String pin = input.nextLine();

        if(saldo < 50000) {
            System.out.println("saldo awal minimal
Rp50.000");

        }else {
            String pin1 = "";
            boolean pinValid = false;

            while(!pinValid) {
                System.out.print("Masukkan PIN (6 digit,
tanpa angka berulang/berurutan): ");
                pin1 = input.nextLine();

                if(Rekening.isValidPin(pin1)) {
                    pinValid = true;
                } else {
                    System.out.println("PIN tidak
valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
dua angka yang berurutan (misal: 12, 54). Silakan coba lagi.\n");
                }
            }

            System.out.print("Pilih Produk: 1. Tabungan Umum \n
2. Giro Bisnis \n 3. Tabungan VIP");
            System.out.print("Pilihan: ");
            int produk = input.nextInt();
            input.nextLine();

            Rekening rekeningBaru = null;

            if (produk == 1) {
                System.out.print("Masukkan suku bunga (%): ");
                double sukuBunga = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningTabungan(
                    no, nama, saldo, pin1, sukuBunga
                );

                System.out.println("Rekening Tabungan berhasil
dibuat");

            } else if (produk == 2) {
                System.out.print("Masukkan batas overdraft (limit
pinjaman): ");

                double batasOverdraft = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningGiro(
                    no, nama, saldo, pin1, batasOverdraft
                );

                System.out.println("Rekening Giro berhasil dibuat");
            }
        }
    }
}

```

```

        } else {
            System.out.println("Pilihan produk tidak valid!
Pembukaan rekening dibatalkan.");
            break;
        }

        daftarRekening.add(rekeningBaru);
        akunAktif = rekeningBaru;
    }
    break;

    case 2:
        if (akunAktif == null) {
            System.out.println("Error : Mohon Maaf,
Anda belum Memiliki nomor rekening!");
        } else {
            System.out.println("Masukkan nominal
setor: ");

            double setor = input.nextDouble();

            if (setor < 10000) {
                System.out.println("saldo setor
minimal Rp10.000");

            } else {

                akunAktif.setorTunai(setor);
            }
        }
        break;

    case 3:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        } else {
            System.out.print("Masukkan PIN untuk otentikasi: ");
            String pinInput = input.nextLine();

            if (akunAktif.otentikasi(pinInput)) {
                System.out.println("Masukkan nominal tarik tunai:
");

                double tarik = input.nextDouble();
                input.nextLine();
                akunAktif.tarikTunai(tarik);
            } else {
                System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
            }
        }
        break;

    case 4:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");

```

```

        }else {
            akunAktif.cekInformasi();
        }
        break;

    case 5:
        if (daftarRekening.isEmpty()) {
            System.out.println("Belum ada rekening
yang terdaftar!");
        } else {
            System.out.println("DAFTAR REKENING
TERDAFTAR ");
            for (int i = 0; i <
daftarRekening.size(); i++) {
                System.out.println("Pilihan Ke-"
+ (i + 1) + ":" );

                daftarRekening.get(i).cekInformasi();

                System.out.println("=====");
            }

            System.out.print("Pilih nomor indeks
rekening yang ingin diaktifkan (1 - " + daftarRekening.size() + "): ");
            int indeksPilihan = input.nextInt();

            if (indeksPilihan >= 1 && indeksPilihan
<= daftarRekening.size()) {

                akunAktif =
daftarRekening.get(indeksPilihan - 1);
                System.out.println("Berhasil
berganti ke akun pilihan Anda!");
            } else {
                System.out.println("Nomor
pilihan tidak valid!");
            }
        }
        break;

    case 6:
        if (akunAktif == null) {
            System.out.println("Error: Anda belum
membuka rekening!");
        }else {
            System.out.print("Masukkan PIN untuk otentikasi: ");
            String pinInput = input.nextLine();

            if (akunAktif.otentikasi(pinInput)) {
                akunAktif.cetakMutasi();
            } else {
                System.out.println("Akses Ditolak: PIN yang Anda
masukkan salah!");
            }
        }
        break;

```

```

        case 7:
            if (akunAktif == null) {
                System.out.println("Belum ada rekening yang
dibuat!");
                break;
            }

            if (akunAktif instanceof RekeningTabungan) {
                RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;
                tabungan.tambahBungaAkhirBulan();
            } else {
                System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan.");
            }
            break;

        case 0:
            isRunning = false;
            System.out.println("Sistem ditutup. Terima
kasih!");
            break;

        default:
            System.out.println("Pilihan tidak
valid!");
    }
}

input.close();
}}

public void cetakMutasi() {
    if (riwayatTransaksi.isEmpty()) {
        System.out.println(" Belum ada transaksi");
    } else {
        for (Transaksi trx : riwayatTransaksi) {
            trx.cetakDetail();
        }
    }
}
}

```

ClassMain berfungsi sebagai pengendali program dan titik masuk yang menjalankan seluruh alur interaksi dengan pengguna. Class ini bertugas menampilkan menu pilihan di layar, membaca input yang diketik pengguna, menyimpan seluruh rekening yang pernah dibuat ke dalam sebuah daftar (ArrayList), serta memutuskan objek Rekening mana yang sedang aktif digunakan. Setiap kali pengguna memilih suatu menu, misalnya setor atau tarik tunai, Main tidak memproses transaksi itu sendiri, melainkan memanggil method yang sudah tersedia di classRekening untuk melakukan pekerjaan sebenarnya.

a. public class Main

```

package pekan4;
import java.util.Scanner;
public class Main {
    public static void main (String[]args) {
        Scanner input = new Scanner (System.in);
        ArrayList<Rekening> daftarRekening = new ArrayList<>();
        Rekening akunAktif = null;
        boolean isRunning = true;
        System.out.println("=== SISTEM PERBANKAN MINI ===");

        while (isRunning) {
            System.out.println("\nMenu Utama: ");
            System.out.println("1. Buka Rekening Baru");
            System.out.println("2. Setor Tunai");
            System.out.println("3. Tarik Tunai");
            System.out.println("4. Cek Informasi Rekening");
            System.out.println("5. Ganti Akun Aktif");
            System.out.println("6. Cetak Mutasi (Riwayat)");
            System.out.println("7. Simulasi akhir Bulan (Khusus
Tabungan");

            System.out.println("0. Keluar");
            System.out.println("Pilih Menu: ");

            int pilihan = input.nextInt();
            input.nextLine();
            switch (pilihan) {

```

Method ini adalah titik masuk (entrypoint) program. Di awal method, dibuat objek Scanner untuk membaca input dari keyboard, variabel akunAktif bertipe Rekening yang sengaja diinisialisasi dengan nilai null karena belum ada rekening yang dibuat saat program baru dijalankan, serta variabel boolean isRunning yang menjadi saklar agar program terus berjalan dalam perulangan menu sampai pengguna memilih keluar. Perlu diperhatikan bahwa versi ini berbeda dari versi sebelumnya: di sini tidak ada ArrayList<Rekening>, sehingga program hanya bisa menangani satu rekening aktif pada satu waktu — setiap kali menu 1 dipilih, rekening lama akan otomatis tertimpa oleh rekening baru.

b. Switch-Case

1. Case 1 Buka Rekening Baru

```

        case 1 :
            System.out.println("Masukkan No Rekening: ");
            String no = input.nextLine();
            System.out.println("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.println("Masukkan Saldo Awal: ");
            double saldo = input.nextDouble();
            input.nextLine();
            System.out.println("Masukkan PIN (6 digit angka): ");
            String pin = input.nextLine();

```

```

        if(saldo < 50000) {
            System.out.println("saldo awal minimal
Rp50.000");

        }else {
            String pin1 = "";
            boolean pinValid = false;

            while(!pinValid) {
                System.out.print("Masukkan PIN (6 digit,
tanpa angka berulang/berurutan): ");
                pin1 = input.nextLine();

                if(Rekening.isValidPin(pin1)) {
                    pinValid = true;
                } else {
                    System.out.println("PIN tidak
valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
dua angka yang berurutan (misal: 12, 54). Silakan coba lagi.\n");
                }
            }

            System.out.print("Pilih Produk: 1. Tabungan Umum \n
2. Giro Bisnis \n 3. Tabungan VIP");
            System.out.print("Pilihan: ");
            int produk = input.nextInt();
            input.nextLine();

            Rekening rekeningBaru = null;

            if (produk == 1) {
                System.out.print("Masukkan suku bunga (%): ");
                double sukuBunga = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningTabungan(
                    no, nama, saldo, pin1, sukuBunga
                );

                System.out.println("Rekening Tabungan berhasil
dibuat");

            } else if (produk == 2) {
                System.out.print("Masukkan batas overdraft (limit
pinjaman): ");

                double batasOverdraft = input.nextDouble();
                input.nextLine();

                rekeningBaru = new RekeningGiro(
                    no, nama, saldo, pin1, batasOverdraft
                );

                System.out.println("Rekening Giro berhasil dibuat");

            } else {
                System.out.println("Pilihan produk tidak valid!
Pembukaan rekening dibatalkan.");
            }
        }
    }
}

```

```

        break;
    }

    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}
break;

```

Setelah data dasar dan PIN valid (bagian yang sama dengan pekan sebelumnya, termasuk syarat saldo awal minimal Rp50.000), program meminta pengguna memilih jenis produk. Variabel rekeningBaru dideklarasikan bertipe Rekening dan diisi null. Jika pengguna memilih 1, program meminta suku bunga lalu membuat objek RekeningTabungan; jika memilih 2, program meminta batas overdraft lalu membuat objek RekeningGiro. Penugasan objek subclass ke variabel bertipe superclass ini disebut upcasting dan dilakukan otomatis oleh Java. Jika pilihan produk selain 1 dan 2, pembukaan rekening dibatalkan dengan perintah break. Rekening yang berhasil dibuat ditambahkan ke daftarRekening dan langsung dijadikan akunAktif.

Output:

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
12345
Masukkan Nama Pemilik:
endy
Masukkan Saldo Awal:
100000
Masukkan PIN (6 digit angka):
123456
Masukkan PIN (6 digit, tanpa angka berulang/berurutan): 208315
Pilih Produk: 1. Tabungan Umum
2. Giro Bisnis
Pilihan:
1
Masukkan suku bunga (%): 5
Rekening atas nama endy berhasil dibuat.
Rekening Tabungan berhasil dibuat

```

2. Case 2 Setor Tunai

```

case2:
    if(akunAktif==null){
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    }else{
        System.out.print("Masukkan nominal
setor: ");

        doublesetor=input.nextDouble();
        akunAktif.setorTunai(setor);
    }

```

```
break;
```

Program terlebih dahulu memeriksa apakah akunAktif masih null. Jika iya, artinya belum ada rekening yang dibuat sehingga transaksi tidak bisa dilakukan dan pesan error ditampilkan. Jika sudah ada rekening aktif, program meminta nominal setor dari pengguna, lalu memanggil methodsetorTunai() milik objek akunAktif — seluruh validasi nominal (minimal setor) diproses di dalam class Rekening, bukan di Main.

Output:

```
=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Error: Mohon maaf, Anda belum memiliki nomor rekening!

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10000.0 berhasil. Saldo saat ini: Rp110000.0
```

3. Case 3 Tarik Tunai

```
case3:
    if(akunAktif==null){
        System.out.println("Error: Mohon maaf,
Anda belum memiliki nomor rekening!");
    }else{
        System.out.println("Masukkan nominal
tarik: ");
        doubletarik=input.nextDouble();
        akunAktif.tarikTunai(tarik);
    }
    break;
```

Alurnya serupa dengan setor tunai: program memastikan ada akun aktif terlebih dahulu, kemudian meminta nominal tarik dari pengguna dan memanggil methodtarikTunai() milik akunAktif. Semua validasi terkait batas minimal dan kecukupan saldo diproses di dalam class Rekening, sehingga class Main hanya berperan meneruskan input pengguna ke method yang bersangkutan.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Error: Mohon maaf, Anda belum memiliki nomor rekening!
```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
9000
Gagal: Minimal tarik 10000!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
10000
Tarik tunai Rp10000.0 berhasil. Saldo anda saat ini: 10000.0

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 3
Masukkan nominal tarik:
11000
| Gagal : Saldo anda tidak cukup!

```

4. Case 4 Cek Informasi Rekening

```

case4:
    if(akunAktif==null){
        System.out.println("Error: Anda belum
membuka rekening!");
    }else{
        akunAktif.cekInformasi();
    }
    break;

```

Program memeriksa apakah akunAktif masih null terlebih dahulu. Jika belum ada rekening yang dibuka, pesan error ditampilkan. Jika sudah ada, methodcekInformasi() milik akunAktif dipanggil untuk menampilkan nomor rekening, nama pemilik, dan saldo akhir ke layar.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
Error: Anda belum membuka rekening!

```

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 4
--- INFO REKENING ---
No. Rekening : 12345
Nama Pemilik : Endy
Saldo Akhir  : Rp10000.0
-----

```

5. Case 5 GantiAkun

```
case5:
    if(daftarRekening.isEmpty()){
        System.out.println("Error: Belum ada
rekening yang terdaftar!");
    }else{
        System.out.println("Masukkan nomor
rekening yang ingin diaktifkan: ");
        String nomorDicari=input.nextLine();

        Rekening ditemukan=null;
        for(Rekening r:daftarRekening){

            if(r.nomorRekening.equalsIgnoreCase(nomorDicari)){
                ditemukan=r;
                break;
            }
        }

        if(ditemukan!=null){
            akunAktif=ditemukan;
            System.out.println("Berhasil
berpindah ke rekening atas nama "+akunAktif.namaPemilik);
        }else{
            System.out.println("Error: Nomor
rekening '"+nomorDicari+"' tidak ditemukan!");
        }
    }
    break;
```

Bagian ini memungkinkan pengguna berpindah ke rekening lain yang sudah terdaftar. Program pertama-tama memeriksa apakah daftarRekening masih kosong; jika kosong, pengguna diberi tahu bahwa belum ada rekening yang bisa dipilih. Jika tidak kosong, pengguna diminta memasukkan nomor rekening yang ingin diaktifkan, kemudian program melakukan pencarian di dalam daftarRekening menggunakan perulangan for. Apabila rekening dengan nomor tersebut ditemukan, akunAktif akan diperbarui ke rekening itu disertai pesan konfirmasi; jika tidak ditemukan, pesan error akan ditampilkan kepada pengguna.

Output:

```
=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
5
Error: Belum ada rekening yang terdaftar!
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun
0. Keluar
Pilih menu:
```

6. Case 6 CetakMutasi

```
Case 6:
        If (akunAktif == null) {
            System.out.println("Error:
Andabelummembukarekening!");
        }else {
System.out.print("Masukkan PIN untuk otentikasi: ");
            String pinInput = input.nextLine();
```

```

        If (akunAktif.otentikasi(pinInput)) {
            akunAktif.cetakMutasi();
        } else {
            System.out.println("AksesDitolak: PIN yang Anda masukkan salah!");
        }
    }
    Break;

```

Menu ini menampilkan riwayat transaksi rekening aktif. Alurnya serupa dengan tarik tunai: program memastikan ada akun aktif, meminta PIN, lalu memanggil `cetakMutasi()` hanya jika PIN benar. Riwayat transaksi bersifat rahasia sehingga perlu autentikasi terlebih dahulu.

Output:

```

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:
6
ID: TRX-S-1790691604005 | Jenis Debit | Nominal: Rp50000.0

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. ganti akun aktif
6. cetak mutasi (riwayat)
7. ganti pin
0. Keluar
Pilih Menu:

```

7. Case 0 Keluar

```

case0:
    isRunning=false;
    System.out.println("Sistem ditutup. Terima
    kasih!");
    break;

```

Saat menu ini dipilih, variabel `isRunning` diubah menjadi `false` sehingga perulangan `while` akan berhenti pada pengecekan kondisi berikutnya, disertai pesan penutup sebagai tanda program selesai dijalankan.

Output:

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
0. Keluar
Pilih menu: 0
Sistem ditutup. Terima kasih!
```

3. Class Transaksi

```
Package pekan3;

Public class Transaksi {
    String idTransaksi;
    String jenis;
    Double nominal;

    Public Transaksi (String id, String jenis, double nominal) {
        This.idTransaksi = id;
        This.jenis = jenis;
        This.nominal = nominal;
    }
    Public void cetakDetail() {
        System.out.println("ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nominal);
    }
}
```

Class `Transaksi` merupakan class baru yang berfungsi sebagai model (cetakan data) untuk satu catatan mutasi pada sebuah rekening. Setiap kali terjadi setoran tunai atau tarik tunai yang berhasil, satu objek `Transaksi` dibuat dan disimpan dalam riwayat rekening. Class ini memiliki tiga atribut, yaitu `idTransaksi` sebagai penanda unik transaksi, `jenis` yang berisi kategori transaksi ("Kredit" untuk uang masuk dan "Debit" untuk uang keluar), serta `nominal` yang menyimpan jumlah uang yang ditransaksikan.

a. Constructor Transaksi

Constructor menerima tiga parameter dan menyimpannya ke atribut objek menggunakan kata kunci `this`. Penggunaan `this` diperlukan karena nama parameter `jenis` dan `nominal` sama dengan nama atributnya, sehingga `this.jenis = jenis` berarti "isi atribut milik objek ini dengan nilai dari parameter".

b. Public void cetakDetail

Method ini menampilkan satu baris rincian transaksi ke layar dengan format ID, jenis, dan nominal. Method ini hanya membaca data dan tidak mengubah apa pun. Dengan menempatkan method pencetak di dalam class Transaksi itu sendiri, objek Transaksi mampu “menjelaskan dirinya sendiri” sehingga class lain cukup memanggil `cetakDetail()` tanpa perlu mengetahui isinya.

4. Class RekeningTabungan

```
package pekan4;

public class RekeningTabungan extends Rekening {
    private double sukuBunga;
    public RekeningTabungan(String nomor, String nama, double
saldoAwal, String pinAwal, double sukuBunga) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.sukuBunga = sukuBunga;
    }

    public void tambahBungaAkhirBulan() {
        double nominalBunga = saldo * (sukuBunga/100);
        saldo += nominalBunga;

        String idTrx = "TRX-B-" + System.currentTimeMillis();
        riwayatTransaksi.add(new Transaksi (idTrx, "Bunga",
nominalBunga));

        System.out.println("Bunga " + sukuBunga + "% berhasil
ditambahkan: Rp" + nominalBunga);
    }
}
```

Class `RekeningTabungan` merepresentasikan rekening tabungan yang memberikan bunga. Kata kunci `extends Rekening` pada deklarasi class menyatakan bahwa `RekeningTabungan` mewarisi seluruh atribut dan method milik `Rekening`, sehingga rekening tabungan otomatis dapat melakukan setor tunai, tarik tunai, cetak mutasi, dan sebagainya tanpa kode tersebut ditulis ulang. Class ini hanya menambahkan hal yang khusus miliknya, yaitu atribut `sukuBunga` dan method `tambahBungaAkhirBulan()`.

a. Constructor RekeningTabungan

Constructor menerima lima parameter, yaitu empat data dasar rekening ditambah `sukuBunga`. Baris `super(nomor, nama, saldoAwal, pinAwal)` memanggil constructor milik superclass `Rekening` untuk mengisi atribut `nomor`, `nama`, `saldo`, dan `PIN`, serta membuat riwayat transaksi. Pemanggilan `super` wajib berada di baris pertama constructor. Setelah bagian induk selesai dibentuk, barulah atribut milik subclass, yaitu `sukuBunga`, diisi. Atribut `sukuBunga` dibuat `private` agar hanya dapat diubah dari dalam class ini.

b. `public void tambahBungaAkhirBulan`

Method ini menghitung dan menambahkan bunga ke saldo. Langkahnya: (1) menghitung `nominalBunga = saldo * (sukuBunga/100)` karena suku bunga dimasukkan dalam persen; (2) menambahkan bunga tersebut ke saldo; (3) membuat ID transaksi berawalan "TRX-B-" lalu mencatat objek Transaksi berjenis "Bunga" ke `riwayatTransaksi`; dan (4) mencetak pesan konfirmasi. Sebagai contoh, pada saldo Rp1.000.000 dengan suku bunga 5%, bunga yang diperoleh adalah Rp50.000 sehingga saldo menjadi Rp1.050.000. Method ini dapat langsung memakai saldo dan `riwayatTransaksi` karena keduanya diwariskan dari `Rekening` dan mengakses `package-private`.

5. Class `RekeningGiro`

```
package pekan4;

public class RekeningGiro extends Rekening {
    private double batasOverdraft;

    public RekeningGiro(String nomor, String nama, double saldoAwal,
String pinAwal, double batasOverdraft) {
        super(nomor, nama, saldoAwal, pinAwal);
        this.batasOverdraft = batasOverdraft;
        // TODO Auto-generated constructor stub
    }
    public double getBatasOverdraft() {
        return batasOverdraft;
    }
}
```

Class `RekeningGiro` merepresentasikan rekening giro bisnis yang memiliki batas overdraft, yaitu batas pinjaman yang boleh digunakan ketika saldo tidak mencukupi. Seperti `RekeningTabungan`, class ini memakai `extends Rekening` dan memanggil `super` pada constructor untuk mengisi data dasar, kemudian menyimpan `batasOverdraft` sebagai atribut khusus. Atribut tersebut bersifat `private` dan dapat dibaca melalui method getter `getBatasOverdraft()`.

Pada versi ini batas overdraft baru disimpan dan belum dipakai dalam proses penarikan, karena method `tarikTunai()` yang dipakai masih milik `Rekening`, saldo tidak boleh kurang dari nominal tarik.

B. TugasPraktikum

Tugas 1:

Pembuatan Multi-Produk (40%) Modifikasi case 1: Buka Rekening Baru di `Main.java`.

1. Setelah user memasukkan Saldo Awal dan PIN, berikan opsi produk: "Pilih Produk: 1. Tabungan Umum | 2. Giro Bisnis"
2. Jika user memilih Tabungan, minta input sukuBunga dalam persen, lalu instansiasi objek RekeningTabungan.
3. Jika user memilih Giro, minta input batasOverdraft (limit pinjaman), lalu instansiasi objek RekeningGiro.
4. Simpan objek tersebut ke dalam daftarRekening (Upcasting: Subclass otomatis dikenali sebagai tipe Superclass-nya).

1. Case 1 Buka Rekening Baru

```

        case 1 :
            System.out.println("Masukkan No Rekening: ");
            String no = input.nextLine();
            System.out.println("Masukkan Nama Pemilik: ");
            String nama = input.nextLine();
            System.out.println("Masukkan Saldo Awal: ");
            double saldo = input.nextDouble();
            input.nextLine();
            System.out.println("Masukkan PIN (6 digit angka): ");
            String pin = input.nextLine();

            if(saldo < 50000) {
                System.out.println("saldo awal minimal
Rp50.000");
            }else {
                String pin1 = "";
                boolean pinValid = false;

                while(!pinValid) {
                    System.out.print("Masukkan PIN (6 digit,
tanpa angka berulang/berurutan): ");
                    pin1 = input.nextLine();

                    if(Rekening.isValidPin(pin1)) {
                        pinValid = true;
                    } else {
                        System.out.println("PIN tidak
valid! Syarat: 6 digit angka, tidak ada angka berulang, dan tidak ada
dua angka yang berurutan (misal: 12, 54). Silakan coba lagi.\n");
                    }
                }

                System.out.print("Pilih Produk: 1. Tabungan Umum \n
2. Giro Bisnis \n 3. Tabungan VIP");
                System.out.print("Pilihan: ");
                int produk = input.nextInt();
                input.nextLine();

                Rekening rekeningBaru = null;

                if (produk == 1) {
                    System.out.print("Masukkan suku bunga (%): ");
                    double sukuBunga = input.nextDouble();

```

```

        input.nextLine();

        rekeningBaru = new RekeningTabungan(
            no, nama, saldo, pin1, sukuBunga
        );

        System.out.println("Rekening Tabungan berhasil
dibuat");

    } else if (produk == 2) {
        System.out.print("Masukkan batas overdraft (limit
pinjaman): ");

        double batasOverdraft = input.nextDouble();
        input.nextLine();

        rekeningBaru = new RekeningGiro(
            no, nama, saldo, pin1, batasOverdraft
        );

        System.out.println("Rekening Giro berhasil dibuat");

    } else {
        System.out.println("Pilihan produk tidak valid!
Pembukaan rekening dibatalkan.");
        break;
    }

    daftarRekening.add(rekeningBaru);
    akunAktif = rekeningBaru;
}
break;

```

Setelah data dasar dan PIN valid (bagian yang sama dengan pekan sebelumnya, termasuk syarat saldo awal minimal Rp50.000), program meminta pengguna memilih jenis produk. Variabel rekeningBaru dideklarasikan bertipe Rekening dan diisi null. Jika pengguna memilih 1, program meminta suku bunga lalu membuat objek RekeningTabungan; jika memilih 2, program meminta batas overdraft lalu membuat objek RekeningGiro. Penugasan objek subclass ke variabel bertipe superclass ini disebut upcasting dan dilakukan otomatis oleh Java. Jika pilihan produk selain 1 dan 2, pembukaan rekening dibatalkan dengan perintah break. Rekening yang berhasil dibuat ditambahkan ke daftarRekening dan langsung dijadikan akunAktif.

Output:

```

=== SISTEM PERBANKAN MINI ===
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
1
Masukkan No Rekening:
12345
Masukkan Nama Pemilik:
endy
Masukkan Saldo Awal:
100000
Masukkan PIN (6 digit angka):
123456
Masukkan PIN (6 digit, tanpa angka berulang/berurutan): 208315
Pilih Produk: 1. Tabungan Umum
2. Giro Bisnis
Pilihan:
1
Masukkan suku bunga (%): 5
Rekening atas nama endy berhasil dibuat.
Rekening Tabungan berhasil dibuat

```

Tugas 2: Simulasi Waktu Akhir Bulan : 60%)

1. Tambahkan menu baru: "7. Simulasi Akhir Bulan (Khusus Tabungan)" di CLI.
2. Di dalam case 7:, periksa apakah akunAktif saat ini adalah objek Tabungan menggunakan operator instanceof.
3. Jika akun tersebut benar RekeningTabungan, lakukan Downcasting ke tipe Tabungan, lalu panggil method tambahBungaAkhirBulan().
4. Jika bukan, tolak dengan pesan: "Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan."

2. Case 7 Simulasi Akhir Bulan

```

case 7:
    if (akunAktif == null) {
        System.out.println("Belum ada rekening yang
dibuat!");
        break;
    }

    if (akunAktif instanceof RekeningTabungan) {
        RekeningTabungan tabungan = (RekeningTabungan)
akunAktif;
        tabungan.tambahBungaAkhirBulan();
    } else {
        System.out.println("Gagal: Fitur bunga akhir bulan
hanya berlaku untuk Rekening Tabungan.");
    }
    break;

```

Menu ini menjalankan perhitungan bunga bulanan. Program pertama-tama memastikan sudah ada akun aktif. Selanjutnya, karena akunAktif bertipe Rekening, variabel ini hanya "mengenali" method milik Rekening dan belum mengenali tambahBungaAkhirBulan() yang hanya ada pada RekeningTabungan. Oleh karena itu program memeriksa jenis objek sebenarnya dengan operator

instanceof. Jika objek tersebut adalah RekeningTabungan, dilakukan downcasting (RekeningTabungan) akunAktif ke variabel tabungan agar method khusus tersebut dapat dipanggil. Jika bukan (misalnya RekeningGiro), ditampilkan pesan bahwa fitur ini hanya untuk rekening tabungan. Pemeriksaan instanceof sebelum downcasting mencegah terjadinya ClassCastException.

Output:

```
Pilih Produk: 1. Tabungan Umum
              2. Giro Bisnis
Pilihan:
1
Masukkan suku bunga (%): 5
Rekening atas nama endy berhasil dibuat.
Rekening Tabungan berhasil dibuat

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
7
Bunga 5.0% berhasil ditambahkan: Rp5000.0
```

```
Pilih Produk: 1. Tabungan Umum
              2. Giro Bisnis
Pilihan:
2
Masukkan batas overdraft (limit pinjaman): 10000000
Rekening atas nama endy berhasil dibuat.
Rekening Giro berhasil dibuat

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Akun Aktif
6. Cetak Mutasi (Riwayat)
7. Simulasi akhir Bulan (Khusus Tabungan)
0. Keluar
Pilih Menu:
7
Gagal: Fitur bunga akhir bulan hanya berlaku untuk Rekening Tabungan.
```

C. Link Repositori GitHub

https://github.com/endypardilian/PrakPBO_C_2511531017/tree/pekan4